Estructuras de Control Funciones

IITA 2022

Primero…un pequeño repaso:

- Python…¿Qué es?
- SOFTWARE
- HARDWARE
- LENGUAJE DE PROGRAMACIÓN
- PROGRAMA
- TIPO DE DATO
- VARIABLE

Temas de hoy:

- **Estructuras de datos:** Conjunto o colección de valores (variables, por ejemplo) organizados de una manera específica en la memoria de la computadora.

- **ESTRUCTURAS DE CONTROL**: Se utilizan para estructurar un programa y así hacerlo más fácil de leer, verificar y mantener
  - Secuencial :Una acción sigue a la otra, es decir, la salida de una instrucción es la entrada de la siguiente
  - Repetitiva: Repiten una secuencia de instrucciones un número determinado de veces
  - Selectiva: Se utiliza para tomar decisiones lógicas
    - Simple (if-else)
    - Múltiple (if-elif-else)

Temas de hoy:

- **Comentarios**: Es una buena práctica para que sea más legible nuestro programa.
  - Se usa # para comentarios de una única línea
  - Se usa ‘‘‘---’’’ para comentarios múltiples líneas

- **FUNCIONES**: Dividen al programa en “módulos” más pequeños
  - Se identifican por un nombre y resuelven alguna necesidad
  - Simplifican la resolución de un problema

Estructura de Control Selectiva Simple

→Decisión simple:

if condición:

“lista de instrucciones”

else :

“lista de instrucciones”

Veamos un ejemplo…

```
#Ejemplo:
x=int(input("Ingresar un nro: "))
c=1
if x>0:
    c=c+1
else:
    c=c-1
print (c)
```

Estructura de Control Selectiva Múltiple

→Decisión múltiple:

if condición booleana:

lista de instrucciones

elif condición booleana:

lista de instrucciones

elif condición booleana:

lista de instrucciones

.. ..

else:

lista de instrucciones

```
z = 7

if z > 8:
  print("¡No voy a imprimir!") #esta sentencia no se ejecuta
elif z > 5:
  print("¡Yo lo haré!") #esta sentencia se ejecuta
elif z > 6:
  print("¡Tampoco voy a imprimir!") #esta sentencia no se ejecuta
else:
  print("¡Yo tampoco!") #esta sentencia no se ejecuta
```

Estructura de Control Repetitiva

- Ciclo repetitivo for

```
for variable in range(veces):
        lista de instrucciones
```

→Ejecuta el ciclo tantas veces como el valor que tenga la variable “veces” encerrada en la palabra reservada range

```python
#Ejemplo 2:
suma=0
print ("Ingrese la cantidad de numeros a sumar: ")
n=int(input())
for i in range(n):
    print (f"Ingrese el numero {i+1}: ")
    a=int(input())
    suma=suma+a
print (f"La suma de los {n} números es: {suma}")
```

Observación: La variable i empieza desde el valor 0, por ello en el ejemplo se muestra i+1 para indicar el orden del número que ingresa.

Estructura de Control Repetitiva

- También podemos hacer:

```
for variable in range(vmin,vmax):
      lista de instrucciones
```

→Ejecuta el ciclo tantas veces que indica el resultado de la resta entre la variable **vmax** y la variable **vmin**. Es decir, realiza **vmax-vmin** veces.

```
#Ejemplo 3:
print ("Tabla del 5, desde el 1 hasta el 15")
for i in range(1,16):
    print("5*", i, "es: ", 5*i)
```

Estructura de Control Repetitiva

- También podemos hacer:

```
for variable in range(vmin,vmax,paso):
        lista de instrucciones
```

→ Ejecuta el ciclo desde el vmin hasta (vmax-1) incrementándose de acuerdo a la variable definida en el paso.

```
#Ejemplo 4:
print("Tabla del 2 solo de los número pares desde el 1 hasta el 9")
for i in range (1,10,2):
    print("2*",i,"es: ", 2*i)
```

Estructura de Control Repetitiva

- Existen los llamados condicionales con evaluación al principio:

```
while condición:
        Lista de instrucciones
```

→Ejecuta el ciclo **MIENTRAS** la condición sea verdadera

```
x=10
while x>0:
    print (x)
    x-=1
```

Estructura de Control Repetitiva

- Existen los llamados condicionales con evaluación al final:

```
while True:
  Lista de instrucciones
  if condicion:
      break
```

→Ejecuta el ciclo **HASTA** que la condición sea verdadera.
El ciclo se ejecuta por lo menos una vez

```
#Ejemplo 6
x=10
while True:
    print (x)
    x-=1
    if x==0:
        break
```

A practicar un poco…

- 1. Solicitar al usuario un número de cliente. Si el número es el 1000, imprimir "Ganaste un premio“

- 2. Solicitar al usuario que ingrese dos números y mostrar cuál de los dos es menor. No considerar el caso en que ambos números son iguales.

- Crear un ciclo infinito, es decir, un ciclo que nunca termina (las instrucciones dentro del ciclo quedan a criterio de cada uno)

A practicar un poco…

4. Escriba un programa que pregunte una y otra vez si desea terminar el programa, salvo si se contesta exactamente SI (en mayúsculas y sin tilde).

5. Haz una tabla de multiplicar utilizando el ciclo ‘for’

6. Cree un bucle que sume los números del 100 al 200

FUNCIONES

- La definición de funciones se realiza mediante la instrucción def más un nombre de función descriptivo -para el cuál, aplican las mismas reglas que para el nombre de las variables- seguido de paréntesis de apertura y cierre.
- Como toda estructura de control en Python, la definición de la función finaliza con dos puntos (:) y el algoritmo que la compone, irá identado:

```
def mi_funcion():
        # aquí el algoritmo
```

FUNCIONES

- Atención! → Una función, no es ejecutada hasta tanto no sea invocada. Para invocar una función, simplemente se la llama por su nombre

```
def mi_funcion():
        # aquí el algoritmo


mi_funcion()
```

- Cuando una función haga un retorno de datos, **estos pueden asignarse a una variable:**

```
def funcion():
        return “Hola mundo”

Mensaje=funcion()
Print(Mensaje)
```

Funciones: Parámetros

- Un parámetro es un valor que la función espera recibir cuando sea llamada
- Una función puede esperar uno o más parámetros

```
def mi_funcion(nombre, apellido):
        print(“Nombre: ”, nombre)
        print(“Apellido: ”, apellido)
```

→ Atención!

Los parámetros que una función espera, serán utilizados por ésta dentro de su algoritmo, a modo de variables de **ámbito local**

Funciones: Parámetros

- Por ejemplo, si tenemos una función:

```
def mi_funcion(nombre, apellido):
        nombre_completo = nombre + “ ” + apellido
        print(nombre_completo)

mi_funcion(‘Nico’, ‘Hussein’)
```

- Si quisiéramos acceder a esas variables locales, obtendríamos un error

Funciones: cuestiones a considerar…

- Python maneja funciones y los procedimientos de la misma forma, pero NO son lo mismo. En la teoría:
  - Función: devuelve un valor
  - Procedimiento: solo ejecuta comandos

- Las funciones en Python pueden devolver más de un valor:

```
def funcion(par1,par2):
    return par1,par2

valor1,valor2=funcion('perro','gato')
print(valor1)
print(valor2)
```

Funciones: cuestiones a considerar…

- Al llamar a una función, siempre se le deben pasar sus argumentos en el mismo orden en el que los espera
- Parámetros por omisión:

```
def saludar(nombre, mensaje = 'Bienvenido!'):
    print(mensaje, nombre)

saludar('Nico')
saludar('Nico', 'Como estas?')
```

- Al asignar parámetros por omisión, no debe dejarse espacios en blanco ni antes ni después del signo =

Funciones útiles de Python

- len(): Calcula la cantidad de caracteres de un string
- find(): Regresa el índice correspondiente al primer carácter de la cadena original que coincide con lo que estamos buscando
- upper() y lower(): convierte a todos los caracteres de una cadena de texto a mayúsculas o a minúsculas
- strip(), lstrip() y rstrip(): para eliminar todos los espacios en blancos, sólo los que aparecen a la izquierda y sólo los que aparecen a la derecha, respectivamente…
- Para obtener substring podemos:
  - Cadena[:valor numerico] → extrae tantos caracteres desde la izquierda como indica el valor num
  - Cadena[valor numérico:] → extrae tatnos caracteres desde el valor numérico hacia el final del texto
  - Cadena[valor numérico1:valor numérico:2] → extrae caracteres desde el valor numérico 1 hasta (valor numerico2 – 1)

Practiquemos sobre lo visto…

- 7. Escriba una función que muestre todos los números primos entre 1 y un número n que es ingresado por parámetro.

- 8. Solicitar al usuario que ingrese su dirección email. Imprimir un mensaje indicando si la dirección es válida o no, valiéndose de una función para decidirlo. Una dirección se considerará válida si contiene el símbolo "@".
- (find) →
- 9. Definir una función que muestre el factorial de un número

- 10. Escriba una función que le pida al usuario ingresar condimentos para un sándwich, hasta que el usuario ingrese ‘salir’. Cada vez que se ingrese un condimento, muestre un mensaje avisando que ya se agregó el condimento al sándwich.